Вичерпний перебір. Задача про рюкзак методом грубої сили

ПІДГОТУВАЛА СТУДЕНТКА КМ-52
ГАБАРАЄВА АННА АНДРІЇВНА

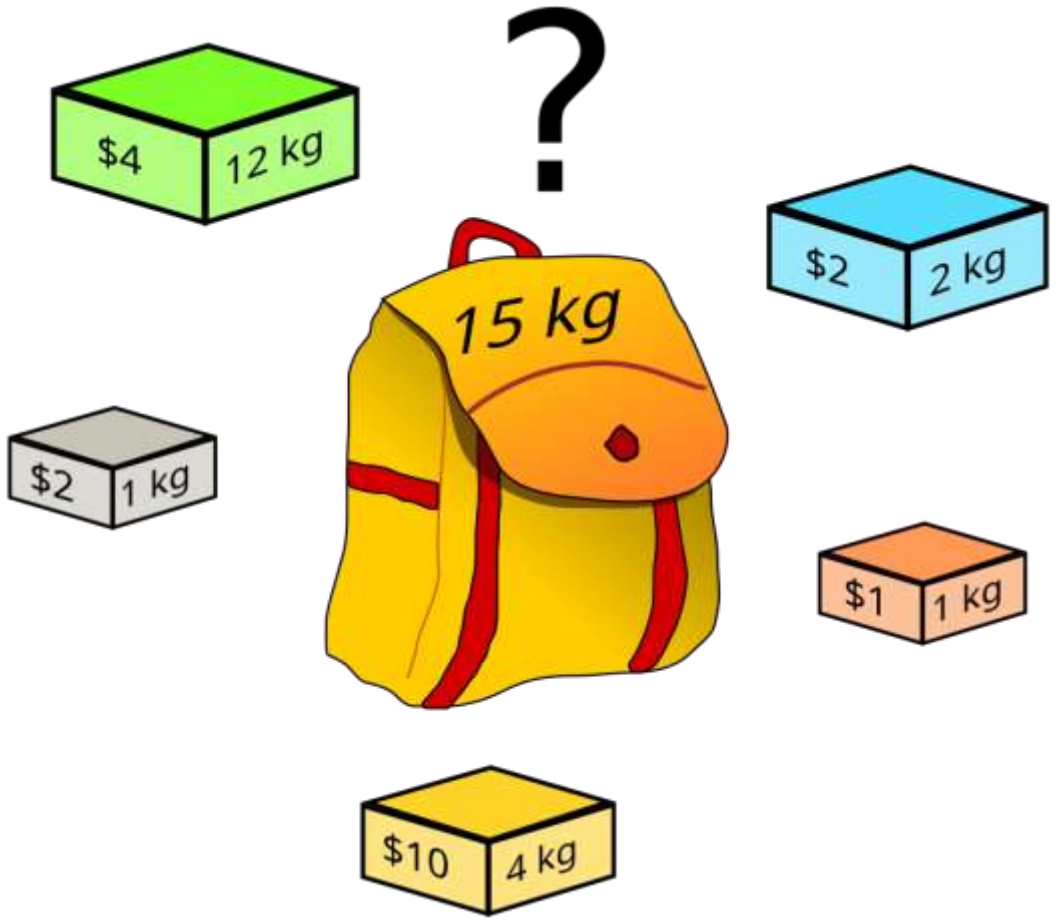
2026 РІК

Основні терміни

- **Вичерпний перебір** - це підхід з використанням грубої сили для вирішення комбінаторних завдань. Він припускає генерацію усіх комбінаторних об'єктів задачі, вибір серед них тих, які задовольняють обмеженням з умов задачі, і подальший пошук потрібного об'єкта.
- **Метод грубої сили** - це прямолінійний підхід до розв'язання задач, який полягає в систематичному переборі всіх можливих варіантів до знаходження правильного рішення.

Задача

Для заданої множини предметів, кожен з яких має вагу і цінність, визначити яку кількість кожного з предметів слід взяти, так, щоб сумарна вага не перевищувала задану, а сумарна цінність була максимальною.



Алгоритм розв'язання

1. Створити всі можливі комбінації з наявних предметів
2. Підрахувати вагу кожної підмножини
3. Виокремити варіанти, які відповідають умові
4. Визначити ціну кожної підмножини з виокремлених варіантів
5. Обрати максимальне значення

Приклад реалізації мовою Python

- Для початку, задамо вхідні дані: 5 предметів та обмеження у вазі - 15
- Для цього використаємо словник:

```
items = {1:[12, 4], 2:[2, 2], 3:[1, 2], 4:[1, 1], 5:[4, 10]} #ключ:[вага, ціна]
max_capacity = 15 #максимальна вага рюкзака
```

Приклад реалізації мовою Python

- 1. Генеруємо всі можливі комбінації через combinations()

```
all_combinations = []
for size in range(len(items) + 1):
    for comb in combinations(items.keys(), size):
        all_combinations.append(comb)
```

Приклад реалізації мовою Python

- 2-3. Визначаємо вагу та відбираємо задовільні варіанти

```
valid_comb = []
for current_comb in all_combinations:
    current_weight = sum(items[item][0] for item in current_comb)
    if current_weight <= max_capacity:
        valid_comb.append(current_comb)
```

Приклад реалізації мовою Python

- 4. Визначаємо ціну кожної відібраної підмножини

```
comb_val = []
for current_comb in valid_comb:
    current_val = sum(items[item][1] for item in current_comb)
    comb_val.append(current_val)
```

Приклад реалізації мовою Python

- 5. Обираємо максимальне значення

```
max_val = max(comb_val)
index = comb_val.index(max_val)
best_comb = valid_comb[index]
```

Приклад реалізації мовою Python

- Таким чином, отримуємо розв'язок

```
Усі можливі комбінації: [(), (1,), (2,), (3,), (4,), (5,), (1, 2), (1, 3), (1, 4), (1, 5), (2, 3), (2, 4), (2, 5), (3, 4), (3, 5), (
4, 5), (1, 2, 3), (1, 2, 4), (1, 2, 5), (1, 3, 4), (1, 3, 5), (1, 4, 5), (2, 3, 4), (2, 3, 5), (2, 4, 5), (3, 4, 5), (1, 2, 3, 4), (
1, 2, 3, 5), (1, 2, 4, 5), (1, 3, 4, 5), (2, 3, 4, 5), (1, 2, 3, 4, 5)]
--------------------
Комбінації, що не перевищують дозволену вагу рюкзака: [(), (1,), (2,), (3,), (4,), (5,), (1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (2,
 5), (3, 4), (3, 5), (4, 5), (1, 2, 3), (1, 2, 4), (1, 3, 4), (2, 3, 4), (2, 3, 5), (2, 4, 5), (3, 4, 5), (2, 3, 4, 5)]
--------------------
Ціна кожної з цих комбінацій:[0, 4, 2, 2, 1, 10, 6, 6, 5, 4, 3, 12, 3, 12, 11, 8, 7, 7, 5, 14, 13, 13, 15]
--------------------
Найкраща комбінація: предмети (2, 3, 4, 5) та їх ціна 15
```

Складність алгоритму та підсумок

- Кожен предмет має два стани - взяти або не взяти. Тому, загальна кількість комбінацій дорівнює 2^n, час роботи алгоритму з вичерпним перебором: O(2^n)
- Алгоритм вкрай неефективний - при збільшенні елементів він стає непридатним, адже може призвести до нестачі оперативної пам'яті, тому програма завершиться аварійно

Додаткова інформація

- Тут я хочу навести посилання на альтернативні методи розв'язання даної задачі:
  - Порівняння методу грубої сили та динамічного програмування: https://ua.linkedin.com/pulse/knapsack-problem-brute-force-vs-dynamic-programming-ouma-winnie-8fvtf?tl=uk
  - Розв'язок задачі за допомогою жадібного алгоритму, але він застосовується виключно, коли предмети можна ділити на частини (Fractional Knapsack Problem). Метод обирає предмети з найвищою питомою цінністю на одиницю ваги, сортуючи їх за цим показником і додаючи по черзі: https://www.youtube.com/watch?v=oTTzNMHM05I
  - Розв'язання через метод гілок і меж. Цей алгоритм знаходить точний розв'язок, послідовно розгалужуючи множину допустимих рішень і відсікаючи ті, що не можуть містити кращого результату: https://www.youtube.com/watch?v=yV1d-b_NeK8